流量治理：多层闸门的业务视角

TokenBucket / SlidingWindow / CircuitBreaker / 异步削峰 / HTTP cache

gomall · 业务侧 deck (v2)

今日大纲

第一层：TokenBucket 全局兜底

第二层：SlidingWindow 用户精确限流

第三层：CircuitBreaker 保护慢依赖

第四层：异步下单削峰

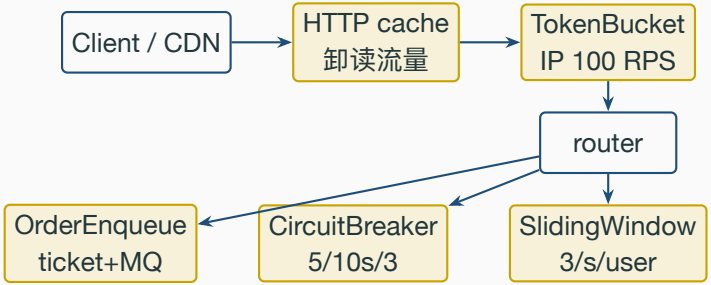
第五层：HTTP cache 把读路径 CDN 化

流量治理是一个” 分层叠加” 的工程

- 单一阈值挡不住所有场景：
  - DDoS 几十万 QPS，网关层就要拦；
  - 黑产用户 5 QPS 持续刷一天，要在用户维度拦；
  - 第三方支付 30s 超时，要在依赖维度拦。
- gomall 把治理拆成 5 层叠加：**TokenBucket / SlidingWindow / CircuitBreaker / 异步削峰 / HTTP cache**。
- 每一层解决一个具体的业务问题，组合起来才能扛住大促。

routes/router.go:22 全局 / *:40 HTTP cache* / *:94* 异步入队 / *:114* 熔断 / *:136* 滑动窗口

gomall 五层闸门的业务定位



五层并不是” 流水线串联”，而是按接口按业务分级**挑选挂载**。*routes/router.go* 全文 *mount* 点

核心 P0 vs 营销 P2：阈值取舍完全不同

等级	接口举例	限流姿态	熔断
P0 交易	/orders/create, /paydown	不挂用户限流	必须有
P1 读	/product/show, /carousels	HTTP cache	不需要
P2 秒杀	/skill_product/skill	用户 3/s 激进	不需要
P2 红包	/redpacket/claim	用户 3/s	不需要
P3 信息	/category/list, 轮播图	全局 + CDN	不需要

P0 宁可下游慢、不能

上游 429；P2 宁可误杀、不能让黑产薅光。

两类接口同一套阈值是行不通的。*routes/router.go:22, 39–45, 76–84, 92, 114–119, 136–142*

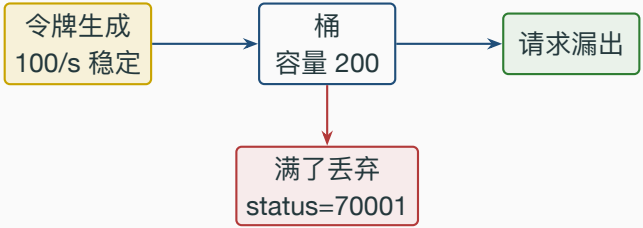
第一层：TokenBucket 全局兜底

为什么全局还要 100 RPS / 200 burst

- gomall 单机基线 **64,254 RPS**（`/ping`），单 IP 真正用户用不到 1/600。
- 100 RPS / 200 burst 实际拦的是**爬虫脚本和异常客户端**，不是用户。
- burst 给到 2× rps 的依据：
  - 浏览器一个页面 30+ 静态 GET 是常态，瞬时 burst ≈ 100；
  - burst 留 200 = 让一次正常加载**绝不触发**。
- 这层是**兜底**：业务层的精细限流挂掉时，至少不会被压死。

routes/router.go:22 TokenBucket(rate.Limit(100), 200)

TokenBucket 算法：桶 + 令牌生成 + 漏出



- 令牌按 **rps** 匀速生成；桶满则溢出。
- 请求拿到令牌才放行，拿不到立刻返回 70001。
- burst 容量决定能否吸收**瞬时尖刺**（比如页面首屏并发）。

middleware/ratelimit.go:22–49 golang.org/x/time/rate 实现

TokenBucket 中间件实现 (逐行)

Listing 1: *middleware/ratelimit.go:22–49*

```go
22 func TokenBucket(rps rate.Limit, burst int) gin.HandlerFunc {
23    var mu sync.Mutex
24    limiters := make(map[string]*rate.Limiter)
25    get := func(ip string) *rate.Limiter {
26       mu.Lock(); defer mu.Unlock()
27       if l, ok := limiters[ip]; ok { return l }
28       l := rate.NewLimiter(rps, burst); limiters[ip] = l; return l
29    }
30    return func(c *gin.Context) {
31       if !get(c.ClientIP()).Allow() {
32          c.JSON(200, gin.H{"status": e.ErrRateLimitExceeded})
33          c.Abort(); return
34       }
35       c.Next()
36    }
37 }
```

 map 按 IP 隔离； 每 IP 一个 `rate.Limiter`； `Allow()` 非阻塞； 拒了直接 70001； 进程内 map 是单机代价，多实例要 Redis token bucket。

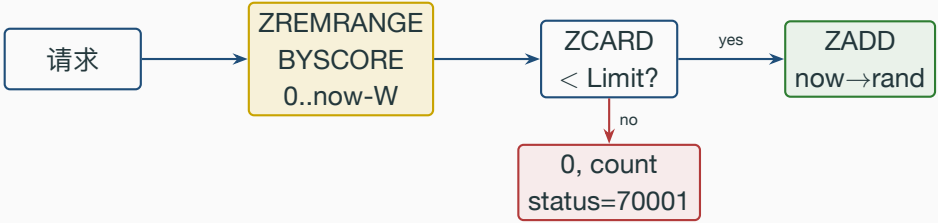
第二层：SlidingWindow 用户精确限流

秒杀为什么必须用 SlidingWindow

- 秒杀的**业务对手**是脚本，不是用户：
  - 真人最快点击 **300–500ms 一次** → 1s 最多 2–3 次；
  - 脚本可以 1s 打 1000 次。
- 阈值定 **3/s/用户**：
  - 真用户：留 50% 余量给” 双击 + 网络重试”；
  - 黑产：第 4 次开始 70001 截断。
- TokenBucket 不够：
  - 必须按**用户 ID**（不是 IP）限，否则一个 NAT 出口下所有用户共享一个桶；
  - 必须**分布式精确**，多实例下用 Redis 共享状态。

routes/router.go:136–142 SlidingWindow Scope:seckill Limit:3 Window:1s ByUser:true

SlidingWindow 在 Redis ZSet 上的实现



- 每个用户一个 ZSet：`rl:seckill:user:42`；
- score = 请求时间戳（ms）；member = ” 时间戳 + 6 字节随机” 避免同毫秒冲突；
- 滑动窗口 = 每次请求先**踢掉 window 之外的旧记录**，再 ZCARD 数当前窗口内的请求数。

repository/cache/ratelimit.go:18–34 Lua 脚本; middleware/ratelimit.go:69–104 调用方

SlidingWindow Lua 脚本 (逐行)

Listing 2: *repository/cache/ratelimit.go:18–34*

```lua
local key, window_ms = KEYS[1], tonumber(ARGV[1])
local limit, now_ms, member = tonumber(ARGV[2]), tonumber(ARGV[3]), ARGV[4]
redis.call('ZREMRANGEBYSCORE', key, 0, now_ms - window_ms)
local count = redis.call('ZCARD', key)
if count >= limit then return {0, count} end
redis.call('ZADD', key, now_ms, member)
redis.call('PEXPIRE', key, window_ms)
return {1, count + 1}
```

逐行：① 入参 key/窗口/阈值/now/请求 id；② **ZREMRANGEBYSCORE** 滑掉过期（关键）；③ ZCARD 数当前窗口；④ 超阈值返回 0；⑤ 通过则 ZADD 入；⑥ PEXPIRE 自动回收；⑦ **整脚本单原子**，无竞态。

压测实测：3/s/user 阈值校验

场景：30 VU × 15s 打 `/skill_product/skill`，共用同一 token。

指标	实测	业务期望
通过请求数	**46**	15s × 3 = 45
被限流数	781,624	黑产全拦
单接口 RPS	52,082	与 ping 同量级
误差	2.2%	< 5%

通过 46≈" 真用户 15s 最多秒杀 45 次"；被限 78 万次说明脚本成本被推到客户端，服务端只做一次 Lua RTT。*stressTest/REPORT.md §5；stressTest/seckill_rate_limit.js*

红包接口：同一套机制再用一次

- `/redpacket/claim` 也是 **Limit:3 Window:1s ByUser:true**（router.go:79–84）。
- 与秒杀不同的是：红包额外挂**幂等**（避免一次抢包重复扣余额）。
- 这印证了一个体例：**SlidingWindow 是 P2 营销接口的” 标配”**，按用户维度反黑产；幂等是 P0/P2 资金链路的” 标配”。
- gomall 把这两个中间件设计成可以**任意组合**，business team 只用写一行声明。

routes/router.go:79–84 红包抢包；*:136–142* 秒杀；*:92, 94, 114, 120* 幂等链

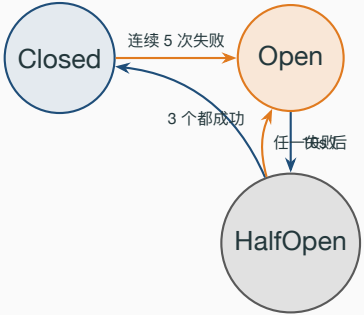
第三层：CircuitBreaker 保护慢依赖

熔断保护的不是自己，是下游

- gomall `/paydown` 的下游 = 第三方支付（Stripe / 微信 / 支付宝）。
- 这种依赖的**失败模式**是慢，不是 500：
  - 正常 RT 200ms；
  - 异常时 RT 30s 才超时；
  - 不熔断 → goroutine 堆积 → 内存爆 → 自己跟着挂。
- CircuitBreaker 的业务含义：**”我代下游回答你不行，免得你等 30 秒”**。
- 三态阈值 **5 失败 / 10s 等待 / 3 探测**（router.go:114–119）：
  - 5 = 容忍偶发抖动，不会一抖就跳；
  - 10s = 给下游一个完整告警 + 自愈周期；
  - 3 = 探测窗口够小，避免 HalfOpen 也被打爆。

routes/router.go:114–119 三态阈值

CircuitBreaker 三态机



Closed：放行所有，连续 5 次失败 → Open。**Open**：直接返回 70002，10s 后试探。**HalfOpen**：放 3 个探测，全成功 → Closed；任一失败 → Open。*middleware/circuitbreaker.go:76–128 allow + report*

CircuitBreaker.allow 状态转移 (逐行)

Listing 3: *middleware/circuitbreaker.go:76–106 allow*

```go
76  func (cb *circuitBreaker) allow() error {
77     switch circuitState(cb.state.Load()) {
78     case stateClosed: return nil // 1) 关闭态零开销
79     case stateOpen:
80        if time.Now().UnixNano()-cb.openedAt.Load() < int64(cb.opt.OpenTimeout) {
81           return errCircuitOpen // 2) Open 期间一律拒
82        }
83        cb.mu.Lock(); defer cb.mu.Unlock()
84        if circuitState(cb.state.Load()) == stateOpen { // 3) 锁内 double-check
85           cb.state.Store(int32(stateHalfOpen))
86           cb.halfOpenReq.Store(0)
87        }
88        fallthrough // 4) 进入 HalfOpen
89     case stateHalfOpen:
90        if cb.halfOpenReq.Add(1) > cb.opt.HalfOpenMaxReq {
91           return errCircuitOpen // 5) 探测超限退回
92        }
93        return nil
94     }
95     return nil
96  }
```

 Closed 仅 atomic load； Open 内不查时间就拒； Open→HalfOpen 锁内 double-check； HalfOpen 用

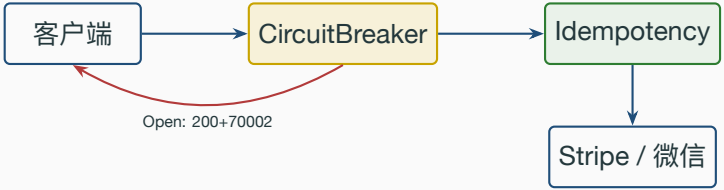
report：决定下一刻是 Closed 还是 Open

Listing 4: *middleware/circuitbreaker.go:108–128 report*

```go
func (cb *circuitBreaker) report(failed bool) {
   switch circuitState(cb.state.Load()) {
   case stateClosed:
      if failed {
         if cb.failures.Add(1) >= cb.opt.FailureThreshold {
            cb.tripOpen() // 连续失败到阈值
         }
      } else { cb.failures.Store(0) } // 成功重置
   case stateHalfOpen:
      if failed { cb.tripOpen() // 任一失败 -> Open
      } else if cb.halfOpenReq.Load() >= cb.opt.HalfOpenMaxReq {
         cb.state.Store(int32(stateClosed)) // 全成功 -> Closed
         cb.failures.Store(0)
      }
   }
}
```

 Closed 失败计数； 成功重置避免偶发抖动跳闸； HalfOpen **非对称**——任一失败立即回 Open，必须全成功才回 Closed； ” 信任成本” 对资金链路合适：宁保守不再打死下游。

熔断 + 幂等的三角配对：Open 返回 200 + 70002



- Open 返回 **200 + status=70002**（不是 5xx）：app 端老网关吞不了 5xx，统一业务码客户端只用一套 switch。
- 客户端拿 70002 指数退避（1s/2s/4s）；重试必须带原 Idempotency-Key。
- paydown 同时挂 CB + Idempotency 不是巧合：缺一边都不行。

routes/router.go:114–121；middleware/circuitbreaker.go:57–65；pkg/e/code.go:58

第四层：异步下单削峰

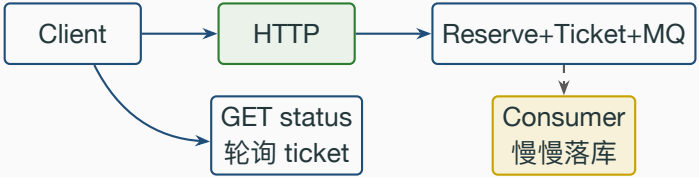
双 11 0 点：100k QPS 怎么扛

- 同步下单链路：HTTP → ReserveStock → INSERT order → COMMIT → 回客户端，p99 大约 50–100ms。
- 100k QPS × 100ms = **1 万并发**进 MySQL，连接池打爆。
- gomall 的方案：把”DB 落库” 从同步链路里拆出来。
  1. 同步部分：reserve 库存 + 写 ticket + 投 MQ → 返回 ticket；
  2. 异步部分：consumer 慢慢消费 MQ → INSERT order → 回写 ticket 状态；
  3. 客户端：拿 ticket 轮询 `/orders/status`。
- 收益：DB 压力从” 瞬时尖刺” 变成” 恒定 1k QPS 平流”。

service/order_async.go:119–172 OrderEnqueue

同步下单 vs 异步 ticket 链路

同步：p99 100ms



上：所有客户端被 DB 阻塞；

下：客户端 5ms 拿到 ticket，DB 压力被 MQ 缓冲。*api 层：EnqueueOrderHandler + OrderStatusHandler; service/order_async.go*

OrderEnqueue：核心 12 行

Listing 5: *service/order_async.go:119–172*

```go
119 func (s *OrderSrv) OrderEnqueue(ctx, req) (interface{}, error) {
120    u, _ := ctl.GetUserInfo(ctx)
121    if err := cache.ReserveStock(ctx, req.ProductID, req.Num); err != nil {
122       return nil, err // 1) 库存预扣
123    }
124    ticket := fmt.Sprintf("%d", snowflake.GenSnowflakeID()) // 2) ticket
125    if err := defaultTicketStore.Put(ctx, ticket,
126       OrderTicketStatus{Status: "pending"}, time.Hour); err != nil {
127       _ = cache.ReleaseReservation(...); return nil, err // 3) 写 Redis
128    }
129    body, _ := json.Marshal(AsyncOrderTask{Ticket: ticket, ...})
130    if err := defaultAsyncProducer.Publish(ctx, body); err != nil {
131       _ = cache.ReleaseReservation(...); return nil, err // 4) 投 MQ
132    }
133    return OrderEnqueueResp{Ticket: ticket, Status: "pending"}, nil
134 }
```

 库存预扣前置，不足直接拒； snowflake 生成 ticket 无 DB； Redis pending 给前端轮询； 任一步失败 **ReleaseReservation** 回滚库存； 投递成功才算入队。

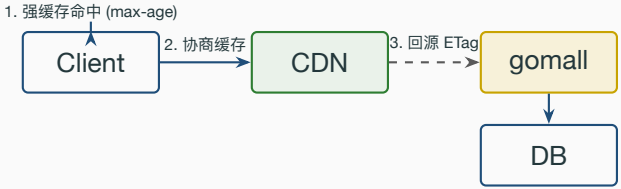
第五层：HTTP cache 把读路径 CDN 化

读流量的最优解：根本不到 gomall

- P1 P3 读接口（`/product/show`, `/carousels`, `/category/list`）的数据更新频率**远低于查询频率**。
- 让浏览器 / CDN 缓存代答：
  - 浏览器：直接返回 304 Not Modified（不传 body）；
  - CDN：命中缓存（不到 gomall）。
- gomall 实现：`HTTPCache(maxAge)` 中间件统一发 ETag + Cache-Control。

middleware/httpcache.go:47–100 HTTPCache; routes/router.go:39–45 挂载点

HTTP cache 的三种命中



- **强缓存**（Cache-Control: max-age=60）：浏览器本地命中，0 RTT；
- **协商缓存**（ETag + If-None-Match）：到 CDN，命中返回 304，无 body；
- **回源**：CDN 不命中，回 gomall；ETag 用 SHA-256 前 16 字节算。

middleware/httpcache.go:78–98 ETag 协商；routes/router.go:39 product/list max-age=30

各接口的 max-age 定多少

接口	max-age	业务依据
/product/list	30s	价格 / 库存可能变化，不能太长
/product/show	60s	商品详情更稳定
/category/list	300s	分类几乎不变
/carousels	300s	运营手动调，时效性弱

max-age 的**业务取舍**：

太长会让” 改价 / 下架” 延迟生效，太短卸不了流量。

300s = 运营改完最多 5min 生效，可接受；30s = 库存接近灵敏阈值的折中。

routes/router.go:39–45 各路由的 *maxAge* 参数

多层流控漏斗：DDoS → IP → 用户 → 业务

- gomall 自己负责 L2 + L3，挡住**绝大部分业务异常流量**。
- L1 / L4 通常由云厂商和独立风控团队提供，不自己造。

routes/router.go:22 (L2); :136–142 (L3)

业务码 70001 / 70002：客服话术对照

业务码	业务含义	客服话术
70001	限流命中	”活动太火爆，请稍后重试”
70002	熔断开启	”支付暂忙，1 分钟后再试”
60002	幂等命中	”您已下单成功，无需重复”
50001	库存不足	”已售罄”

同样是”请稍后”，70001 客户端重试就能过、70002 必须等下游自愈、60002 是用户自己重复了。

区分话术比统一一句”系统繁忙”对用户更友好，对客服也更省事。*pkg/e/code.go:57–58* 业务码；*pkg/e/msg.go:51–52* 消息

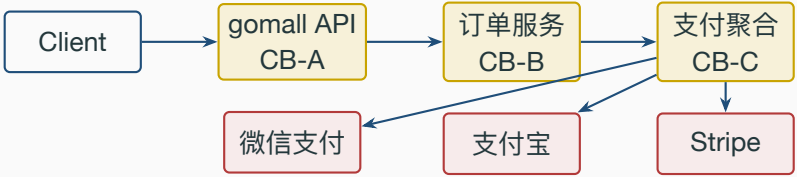
error budget：用数字管” 什么时候停发版”

- P0 SLO=99.95% → 月度**宕机预算 ≈ 21min**。
- 烧 1/3（7min）减发版；烧 2/3（14min）冻结非紧急；全烧光 → 下月转” 稳定性 only”。
- 预算用 SLI 推：限流命中率 / 熔断打开时间 / 5xx 比例累加成” 不可用时间”。

> 流量治理不是**一个机制**，是**五个机制按业务等级组合**。
> 每个阈值都要能用业务语言解释清楚。

Google SRE error budget 原则；*stressTest/REPORT.md* 链路稳定性数据

服务调服务调服务：三层熔断的级联防护



- CB-C 守第三方：单家挂掉只断单家，其余通道照走。
- CB-B 守” 支付聚合服务” 整体：三家全挂时止血，避免订单服务自身被拖。
- CB-A 守” 订单服务” 整体：极端场景下退化到只读，让用户至少能看到订单列表。

middleware/circuitbreaker.go:44–136 单实例 CB; routes/router.go:114–119 仅 CB-A 接入

级联防护的两个隐藏陷阱

- **陷阱 1：阈值同步雪崩**。如果 CB-A 与 CB-B 都设 5/10s，下游恢复瞬间所有上游同时 HalfOpen → 探测流量乘 N 倍打回去 → 下游再次挂。
  对策：外层 CB 的 OpenTimeout **要长于**内层（如 30s vs 10s），错开探测窗口。
- **陷阱 2：兜底链路也会挂**。CB-C Open 时若兜底返回的 cache stale 也过期，CB-B 会把 cache 读失败计作业务失败 → 跟着 Open。
  对策：兜底路径**不能**计入 CB-B 的失败计数；report 时按 status code 维度分桶。
- gomall 当前只挂了 CB-A（paydown），CB-B / CB-C 是把单实例 CB 复制到下游服务上的扩展模式。

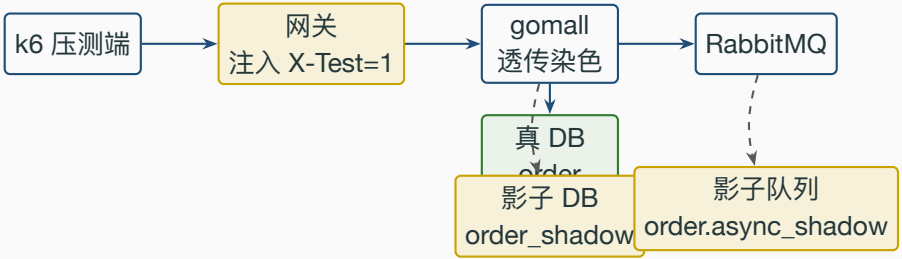
middleware/circuitbreaker.go:69 失败判定 = 5xx 或 c.Errors; 兜底路径需要在 handler 内吞掉 5xx 改成 200+stale

压测的两难：不压不知道、压坏怕影响真用户

- 线下压测的天花板：本地依赖、数据规模、缓存命中率都跟线上不一样，结果偏乐观。
- 真在线上压：每条链路写进 `order` / `stock` 表会污染真实业务数据。
- 全链路压测平台的本质：在**同一套线上代码**里区分”压测流量 vs 真流量”，让前者落进**影子库 / 影子表 / 影子 MQ**。
- 关键技术：**流量染色**（traffic coloring）——染色标识跟着 trace 一路透传到 DAO 层，DAO 看到色就切表。

阿里 / 字节公开过的全链路压测方案；*gomall* 当前未实现，本节为扩展设计

全链路压测：影子库 / 影子 MQ / 流量染色



- DAO 看 ctx 里的 `X-Test` 决定 `order` vs `order_shadow`；
- MQ 按 routing-key 分流，consumer 只消费**对应色**的队列；
- Redis 用 key 前缀 `shadow:` 隔离；缓存命中率与真链路解耦。

扩展设计：*gomall DAO 层（dao/*.go）+ middleware/coloring.go*（待新增）

流量染色中间件：ctx 透传一行入侵

Listing 6: 设计稿：*middleware/coloring.go*

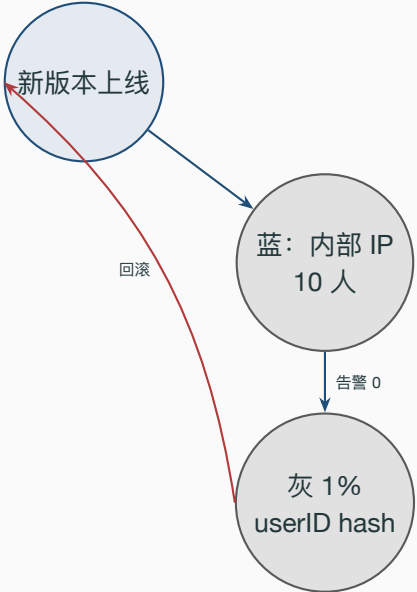
```go
const ColorKey = "x-traffic-color"

func TrafficColoring() gin.HandlerFunc {
   return func(c *gin.Context) {
      color := c.GetHeader("X-Test") // 1) 取染色标识
      if color == "" { color = "prod" }
      ctx := context.WithValue(c.Request.Context(), ColorKey, color)
      c.Request = c.Request.WithContext(ctx) // 2) 写回 ctx
      c.Next() // 3) 后续 DAO 从 ctx 读
   }
}

// DAO 切表（示例）
func tableOrder(ctx context.Context) string {
   if v, _ := ctx.Value(middleware.ColorKey).(string); v == "test" {
      return "order_shadow" // 4) 影子表
   }
   return "order"
}
```

染色只在入口注入一次；ctx 是 trace 天然载体，trace 框架已经在带；DAO 看 ctx 决定表名，零侵入业务代码；缺点：所有 DAO 都要改一行；好处：可以做到**产线压测无副作用**。

红蓝灰发布：按 IP / 按 userID / 按比例的流量切



三种切流维度的业务取舍

切流维度	业务含义	适用场景
按 IP / 内部段	” 公司同事先用”	蓝绿、冒烟、QA 验收
按 userID hash	” 同一用户一定看同一版”	灰度、A/B、新人引导
按比例随机	” 整体 X% 流量”	性能压力、稳定性验证
按地域 / 设备	” 先在某区域 / 设备试”	区域化运营、客户端兼容

- 同一用户切到不同版本 → 购物车 / 订单 / 优惠券容易出现” 看到一半的数据”，要么按 userID hash，要么版本兼容。
- 按比例随机适合” 无状态读路径”（首页、商品详情）；写路径要按 userID 锁定。

网关层选型：Kong / APISIX / Nginx Ingress / 自研

方案	数据面	控制面	限流颗粒	适合
Nginx Ingress	nginx + Lua	K8s CRD	IP / Header	K8s 集群入口
Kong	nginx + Lua	PostgreSQL	服务 / 消费者	中大型微服务
APISIX	nginx + Lua	etcd	服务 / 路由 / Consumer	高动态变更
自研 gateway	Go / Rust	自管	业务字段任意	业务定制极重
gomall 现状	gin in-proc	进程内	IP / 用户	单体直连

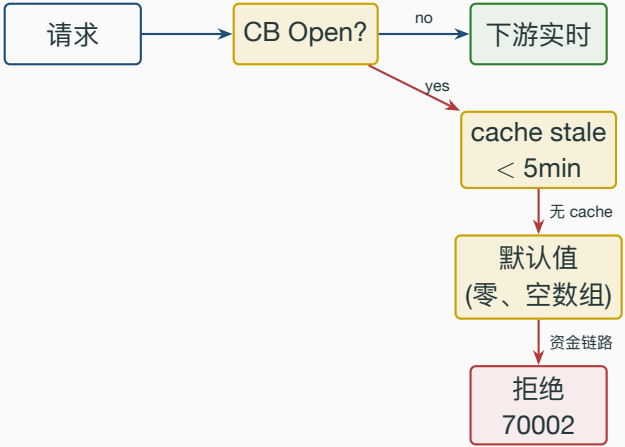
gomall 没接独立网关，限流 / 熔断都跑在 gin 中间件里。**进生产前一般会接 Nginx Ingress + APISIX**：进程内中间件守业务码与状态机正确性，网关守 IP / DDoS / TLS。对照：*APISIX* 公开文档 *limit-req/limit-count* 插件配置

五种主流限流算法：各自的业务边界

算法	适合场景	局限	
计数器 fixed-window	简单兜底 / 日级配额	窗口边界”双倍突刺”	
滑动窗口 sliding-log	精确反黑产	内存 / Redis 开销大	
滑动窗口 sliding-counter	折中	仍有边界误差	gomall 选**滑**
令牌桶 token-bucket	允许突发	不抗持续高水位	
漏桶 leaky-bucket	强制平滑	突发延迟	
自适应 Sentinel BBR	容量自动反推	调参复杂 / 黑盒	

动窗口 ZSet（sliding-log 变种）跑秒杀，因为业务上要**精确反黑产**；选**令牌桶**跑全局，因为允许首屏突发。*middleware/ratelimit.go:22–49 token-bucket; repository/cache/ratelimit.go:18–34 sliding-log*

熔断打开后：兜底数据让”不可用”变”降级可用”



- 读路径（商品 / 推荐 / 评价）：cache stale > 默认值 > 拒绝。用户能看到旧数据胜过白屏。
- 写路径（支付 / 下单 / 扣库存）：**宁可拒，不要默认值**，否则资金 / 库存对不上

业务码全表：错误码 → 客户端行为 → 客服话术

码	含义	客户端	监控位	客服话术
20000	成功	正常	业务 KPI	–
40001	参数非法	不重试	调用方告警	”请检查输入”
40101	未登录	跳登录	不告警	”请先登录”
40301	无权限	不重试	不告警	”无权限”
50001	库存不足	不重试	业务面板	”已售罄”
50002	网络超时	重试 + 幂等	SRE 告警	”网络繁忙、请稍后”
60001	幂等冲突	拉结果	业务面板	”请稍后查询”
60002	幂等命中	读结果	不告警	”您已成功，无需重复”
70001	限流命中	1s 重试	治理面板	”活动太火、请稍后”
70002	熔断开启	指数退避	SRE 告警	”支付暂忙、1 分钟后再试”
70003	黑产识别命中	不重试	风控面板	”为了账户安全、请联系客服”

pkg/e/code.go:57–58 / msg.go:51–52；70003 为扩展位

大促容量评估：怎么算出”需要多少机器”

- 输入：① 业务侧预估的峰值 QPS；② 单机基线（gomall 64k RPS）；③ 安全系数（一般 1.5–2×）。
- 计算：副本数 = ⌈ 预估峰值 × 安全系数 / 单机基线 ⌉。
- 双 11 例：100k QPS / 64k RPS × 1.5 = 2.34 → **至少 3 台**。
- 但秒杀 / 红包链路 **不是** 64k RPS 的水平：实测 52k 是 `/skill_product/skill`，下单写链路 50k；要按**最弱链路**（这里是异步落库的 consumer 消费速度）算容量。
- 容量评估的常见错误：① 只看 RPS 不看 p99；② 没考虑下游放大（一个 paydown 触发 3 个下游调用）；③ 忽略冷启动（K8s 拉镜像 30s 起步）。

stressTest/REPORT.md §1 各链路基线 RPS

HPA 自动扩缩容：怎么定 metric 与冷却时间

- CPU-based HPA 是最朴素的：CPU > 70% 扩、< 30% 缩。问题：扩容前 30s 已经把请求拒了。
- 自定义 metric HPA：用 QPS / 队列堆积量 / 连接数作为触发条件，比 CPU 提前 1–2min 感知。
- gomall 异步下单的天然信号：MQ 队列堆积 > 10k → 扩 consumer。
- 关键：**冷却时间**（cooldown）——缩容窗口必须长于扩容（10min vs 1min），避免抖动来回拉。
- **预热与缓冲池**：新副本起来后先放 5% 流量预热（JIT、连接池、缓存预填），10s 后切到 100%，避免冷启动 p99 暴涨。

扩展设计；*K8s HPA + Custom Metrics API*；*RabbitMQ queue depth* 作为 *metric*

面试 Q&A (1/4)

Q1：100 RPS / 200 burst 这两个数字怎么定?

A：基线 64k RPS / 单 IP 用不到 1/600，100 留给爬虫脚本做兜底；burst 200 容纳一次首屏并发 100+，不会误伤真用户。

Q2：为什么秒杀用 SlidingWindow 不用 TokenBucket?

A：① 必须按用户而不是 IP，否则 NAT 出口全员共享；② 必须分布式精确，多实例下用 Redis 共享状态；③ ZSet 比 fixed-window 没有窗口边界可以利用。

Q3：CircuitBreaker 5/10s/3 三个数字有什么讲究?

A：5 = 容忍偶发抖动不一抖就跳；10s = 给下游一次完整告警 + 自愈周期；3 = HalfOpen 探测够小，避免再次打死下游。

面试 Q&A (2/4)

Q4：异步下单是不是会丢单?

A：不会。库存先 Reserve（Redis Lua 原子），任一失败都 ReleaseReservation 回滚；ticket TTL=1h 兜底；MQ 用 RabbitMQ 持久化 + 手动 ack。代码位置 `service/order_async.go:119--172`。

Q5：为什么熔断返回 200 不返回 503?

A：app 端老网关不能优雅处理 5xx；统一走业务码（70002）= 客户端只用维护一套 status switch；监控不要按 4xx/5xx 算错误率，必须看 status。代码位置 `middleware/circuitbreaker.go:57--65`。

Q6：HTTP cache 的 max-age 怎么定?

A：取决于" 改价 / 下架" 的最大可接受延迟。商品详情 60s，分类 / 轮播 300s，列表 30s（库存敏感）。代码位置 `routes/router.go:39--45`。

面试 Q&A (3/4)

Q7：多层熔断会不会” 探测雪崩”？怎么办？

A：会。外层 CB 的 OpenTimeout 要长于内层（30s vs 10s）错开 HalfOpen 探测窗口；同时兜底路径不应计入失败计数，否则 cache 过期会让外层跟着 Open。代码位置 `middleware/circuitbreaker.go:69` 失败判定。

Q8：全链路压测怎么避免污染真实数据？

A：流量染色：网关层注入 `X-Test` header → 透传到 ctx → DAO 看 ctx 切 `order_shadow` 表；MQ 用独立 routing-key 走影子队列；Redis 用 `shadow:` 前缀隔离。代码位置 `middleware/coloring.go`（扩展）。

Q9：灰度发布按什么维度切流最合理？

A： 内部 IP 段——蓝绿冒烟； userID hash——保证” 同一用户看同一版”，避免购物车看到一半的数据； 按比例随机——只适合无状态读路径。写路径必须按 userID 锁定。

面试 Q&A (4/4)

Q10：限流算法为什么不选自适应（Sentinel BBR）？

A：自适应是黑盒、冷启动 5–10min 才稳；秒杀 3/秒/用户这种阈值用业务语言能算清楚，没必要让运维去看 BBR 曲线判断。代码位置 `repository/cache/ratelimit.go:18--34`。

Q11：HPA 用什么 metric 比 CPU 更合适？

A：异步下单链路用 **MQ 队列堆积**：堆积 > 10k 扩 consumer，比等 CPU 飙高快 1–2min；同步链路看 inflight 请求数 / p99 延迟；cooldown 缩容窗口要长于扩容（10min vs 1min）避免抖动。

代码位置一览

router 挂载点	*routes/router.go:22, 39–45, 79–84, 94, 114–121, 136–142*
TokenBucket	*middleware/ratelimit.go:22–49*
SlidingWindow	*middleware/ratelimit.go:59–106 + repository/cache/ratelimit.go:18–34*
CircuitBreaker	*middleware/circuitbreaker.go:44–136*
HTTPCache	*middleware/httpcache.go:47–100*
OrderEnqueue	*service/order_async.go:119–172*
业务码	*pkg/e/code.go:57–58 / msg.go:51–52*
压测报告	*stressTest/REPORT.md §1, §5*

配套技术 deck：05-ratelimit-circuit-breaker.tex（机制视角）